SERVER:
COMPOSABLE MEMORY
SYSTEMS (CMS)



Disaggregated CXL Memory Architecture for Agentic AI : Dynamic Pooling, Sharing, and PNM Offloading

Jinin So, Kyumin Park, Harry Kim, Junchen Jiang, Arifur Rahman

Samsung, XCENA, Tensormesh, Marvell

SERVER:
COMPOSABLE MEMORY
SYSTEMS (CMS)



CXL Dynamic Memory Pooling for Agentic AI

Kyumin Park, Junchen Jiang

Samsung, Tensormesh

Legal Disclaimer

Copyright ©2013-2026 Samsung Electronics Co., Ltd. Confidential. All rights reserved.

This document has been prepared by Samsung Electronics Co., Ltd. (“Samsung”). All contents in this document is the property of Samsung, protected by applicable laws and non-disclosure agreements between Samsung and you or your employer, as applicable. You are strictly prohibited from, without limitation, disclosing, copying, reproducing, distributing, transmitting, modifying, rebroadcasting, re-encoding, re-presenting, exploiting, or creating derivative works of this document or any parts of this document without the prior written permission from Samsung.

The contents of this document are provided for informational purposes only. No representation or warranty (whether express or implied) is made by Samsung or any of its officers, advisers, agents, or employees as to the accuracy, reasonableness or completeness of the information, statements, opinions, or matters contained in this document, and they are provided on an "AS-IS" basis. Samsung will not be responsible for any damages arising out of the use of, or otherwise relating to, this document. Nothing in this document grants you any license or rights in or to information, materials, or contents provided in this document, or any other intellectual property. All third-party brand names, product names, and trademarks referenced herein are the property of their respective owners. Such references are solely for the purpose of describing testing environments and do not imply any affiliation with, endorsement by, or sponsorship by the respective trademark or copyright holders.

The contents of this document may also include forward-looking statements. These forward-looking statements include all matters that are not historical facts, as well as statements regarding Samsung's intentions, beliefs and current expectations concerning, among other things, market prospects, growth, strategies, and the industry in which Samsung operates. By definition, forward-looking statements involve risks and uncertainties because they relate to events and depend on circumstances that may or may not occur in the future. Samsung hereby reminds you that forward-looking statements are not guarantees of future performance and that the actual developments of Samsung, the market, or the industry in which Samsung operates may differ materially from those made or suggested by the forward-looking statements contained in this document or in the accompanying oral statements. In addition, even if the information contained herein or the accompanying oral statements are shown to be accurate, those statements may not be indicative of future developments.

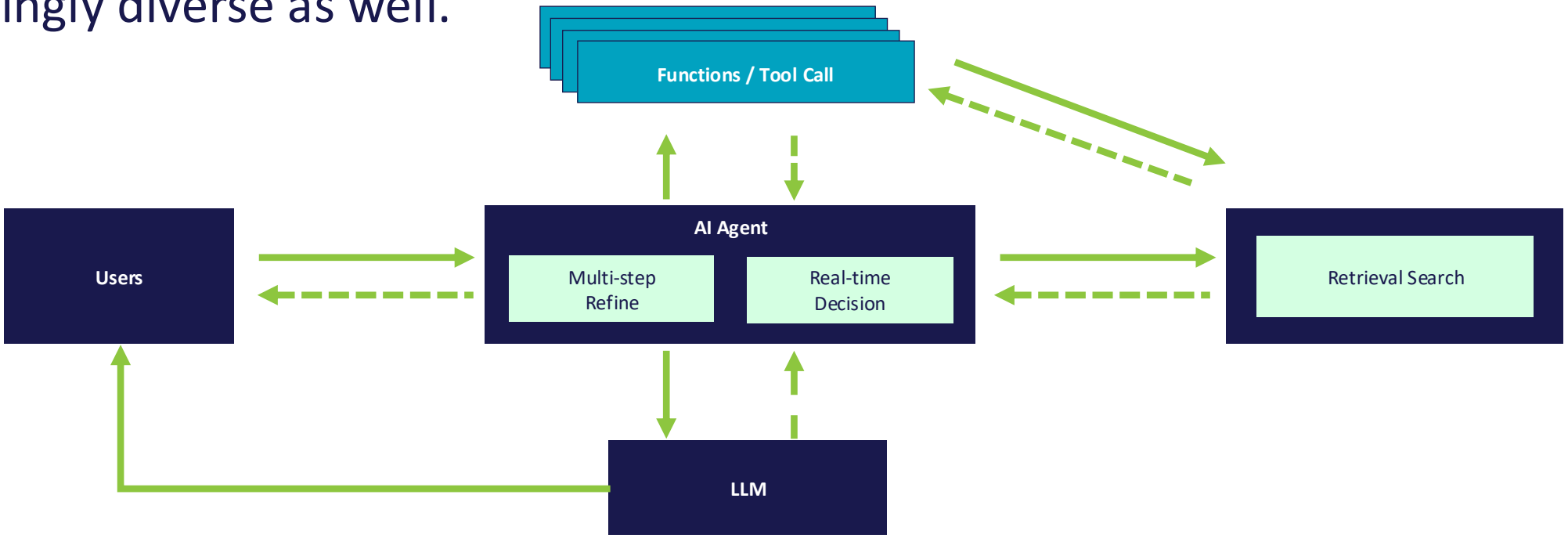
All contents in this document may be subject to change without notice. Without limiting the generality of the foregoing,

1. All design, features and specifications represented herein may change without notice;
2. Images shown here have been adjusted for demonstration purposes and may appear differently on the actual products;
3. All data on products herein, including their performances, are based on internal testing using standard Samsung benchmarks under laboratory conditions. Test results do not guarantee future performance under such test conditions, and the actual throughput or performance that any user will experience may vary depending upon many factors; and
4. All images on screen are simulated, except where otherwise noted.

BY CONTINUING TO ACCESS THIS DOCUMENT, YOU ARE DEEMED TO HAVE READ, UNDERSTOOD, AND AGREED WITH THE FOREGOING TERMS AND CONDITIONS.

Workflow of AI Agentic

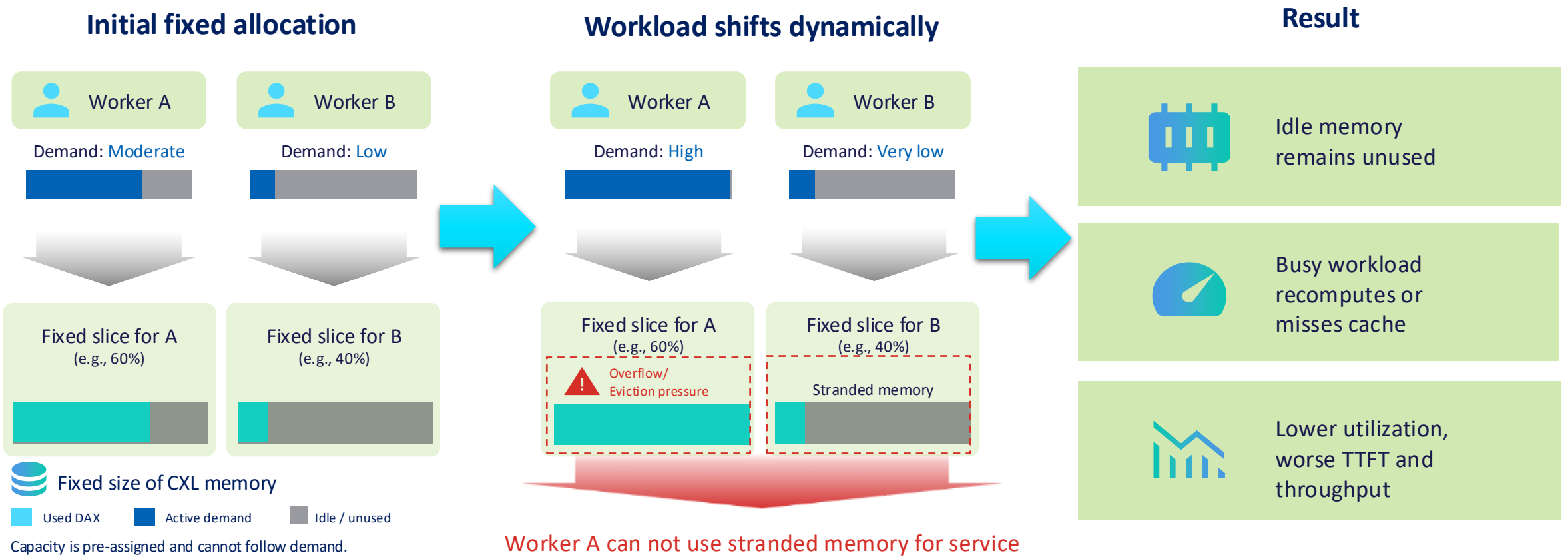
- AI workloads are becoming increasingly iterative and complex, requiring real-time decision-making through CPU-based tool calls.
- As the diversity of tool calls increases, memory requirements are becoming increasingly diverse as well.



Workflow of AI Agentic

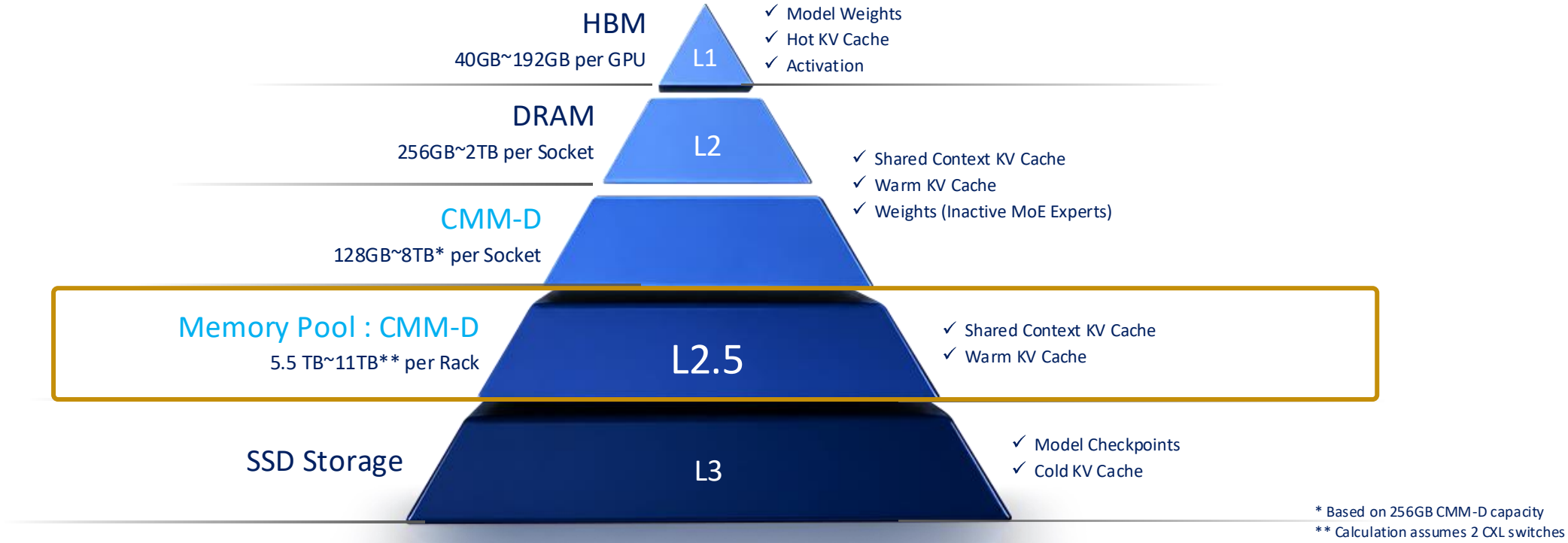
The Problem : Stranded Memory in Fixed Allocation

- AI memory requirements vary significantly across users and workloads



CXL Memory Pooling for dynamic management

- A shared memory tier enabling scalable and elastic memory management



Lower System Cost per Performance

Pooled memory eliminates the need to over-provision GPUs for KV cache, reducing the effective cost of serving long-cont ext requests.

Elastic, Poolable Capacity

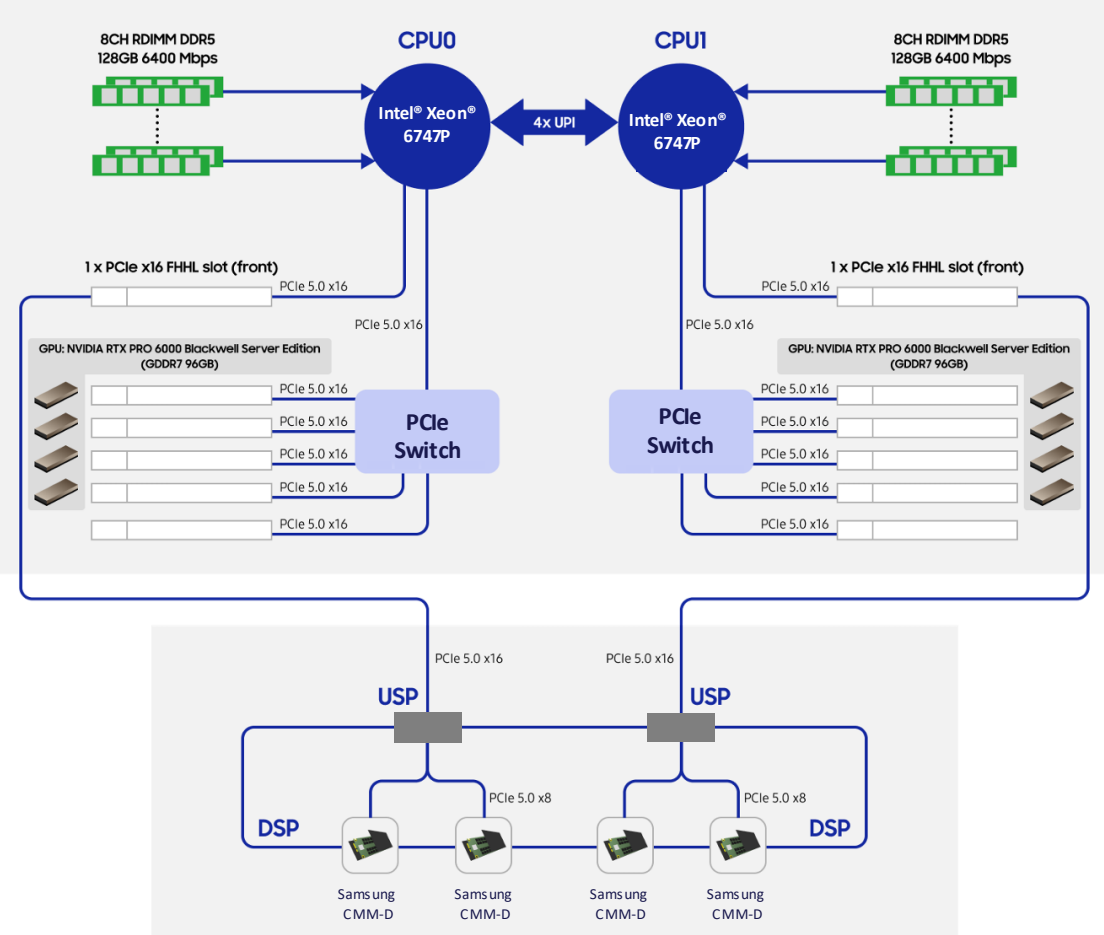
Breaks the GPU HBM capacity wall by exposing memory as a shared, addressable pool across nodes, instead of stranded DRAM on each GPU.

Shared Visibility for Prefill/Decode Disaggregation

Prefill and Decode workers directly access the same KV cache tier, eliminating network hops and re-serialization during hand off.

CXL Memory Pooling System

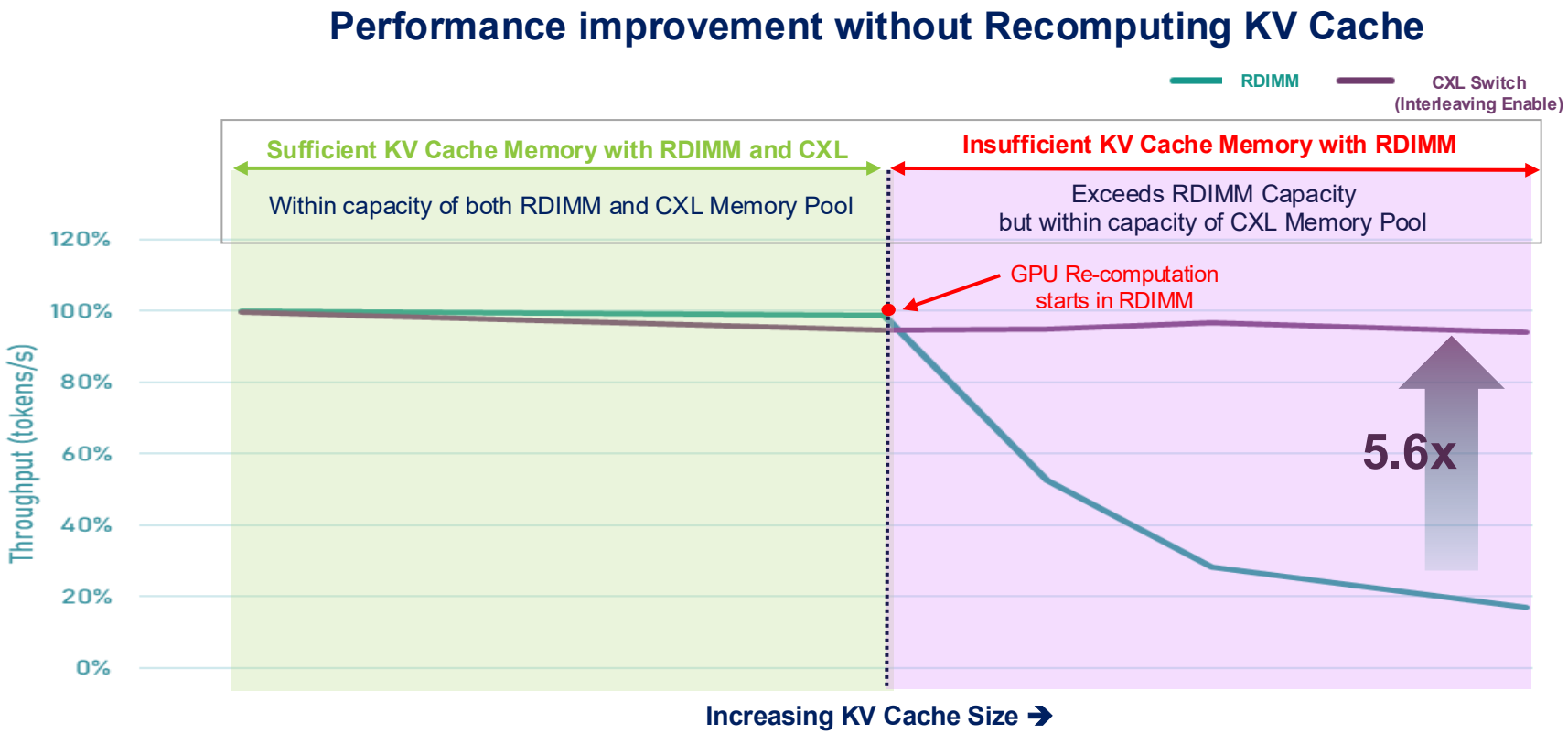
- Blackwell GPU + CXL Memory Pooling system for evaluating performance impact



HW/SW Environment	
CPU	Intel® Xeon® 6747P Processor
GPU	NVIDIA RTX PRO 6000 Blackwell Server Edition (GDDR7 96GB) * 4ea per Socket
DRAM	RDIMM DDR5 128GB 6400Mbps 8CH per Socket
CXL Switch	Marvell CXL 2.0 Switch 1USP - 2DSP (MD220 256GB 6400Mbps * 2ea) per Socket
vLLM ver.	v0.13.0
LMCache ver.	v0.3.12, v0.3.14.dev0 : in-house interleaving-enabled version
OS ver.	Ubuntu 24.04.2 LTS
Kernel ver.	linux 6.14.5 : in-house Virtual NUMA node–enabled version
CUDA ver.	13.0
Model	Llama-3.1 8B : Single GPU, Tensor Parallelism=1 evaluation Llama-4-Maverick-17B-128E-Instruct-FP8 : Multi GPU, Tensor Parallelism=8 evaluation

LLM Performance with CXL Memory Pool

- CXL Memory Pool delivers 5.6x higher sustained performance by eliminating GPU re-computation



Evaluation Configuration

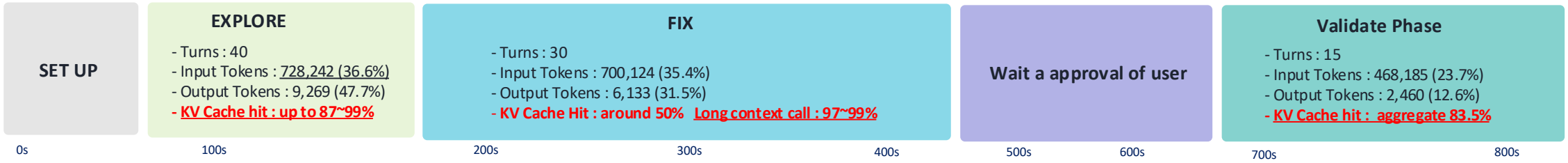
Long_Doc_QA
. vLLM 0.13.0
. LMCache 0.3.14
. Llama3.1 8B FP16
. RTX PRO 6000 8GPU
+ RDIMM 128GB x 8Ch
+ MD220 256GB x 4EA

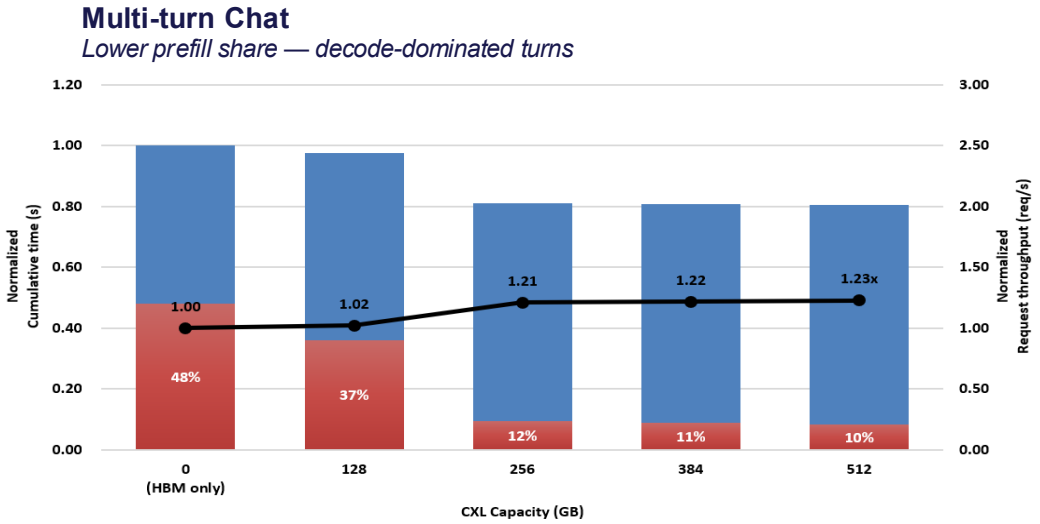
Enhancing Performance for the Emerging Agentic AI

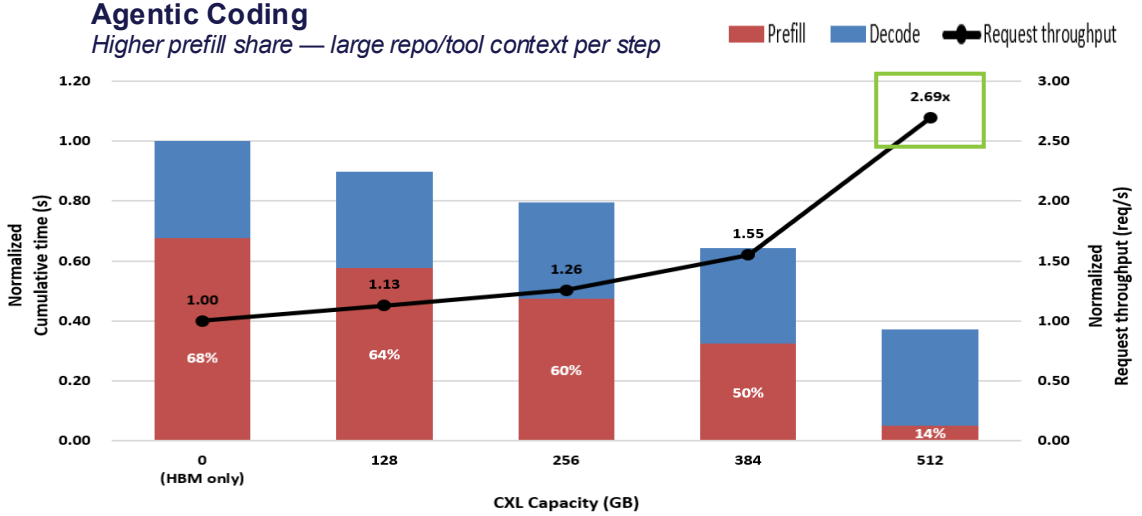
- Agentic AI workloads achieve high KV cache hit rates, increasing the effectiveness of CXL pool-based KV cache offloading

Analysis of Agentic Coding workload

Using Claude code(sonnet 4.5) scenario in lmcache-agentic-traces : Hugging face

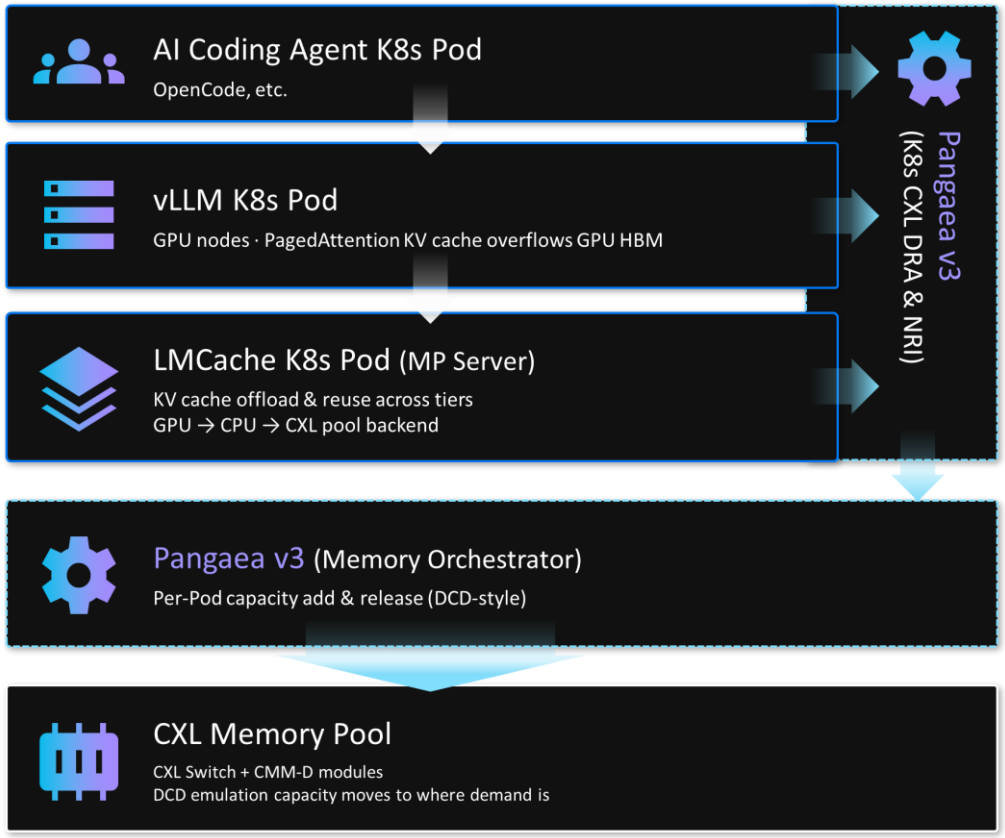


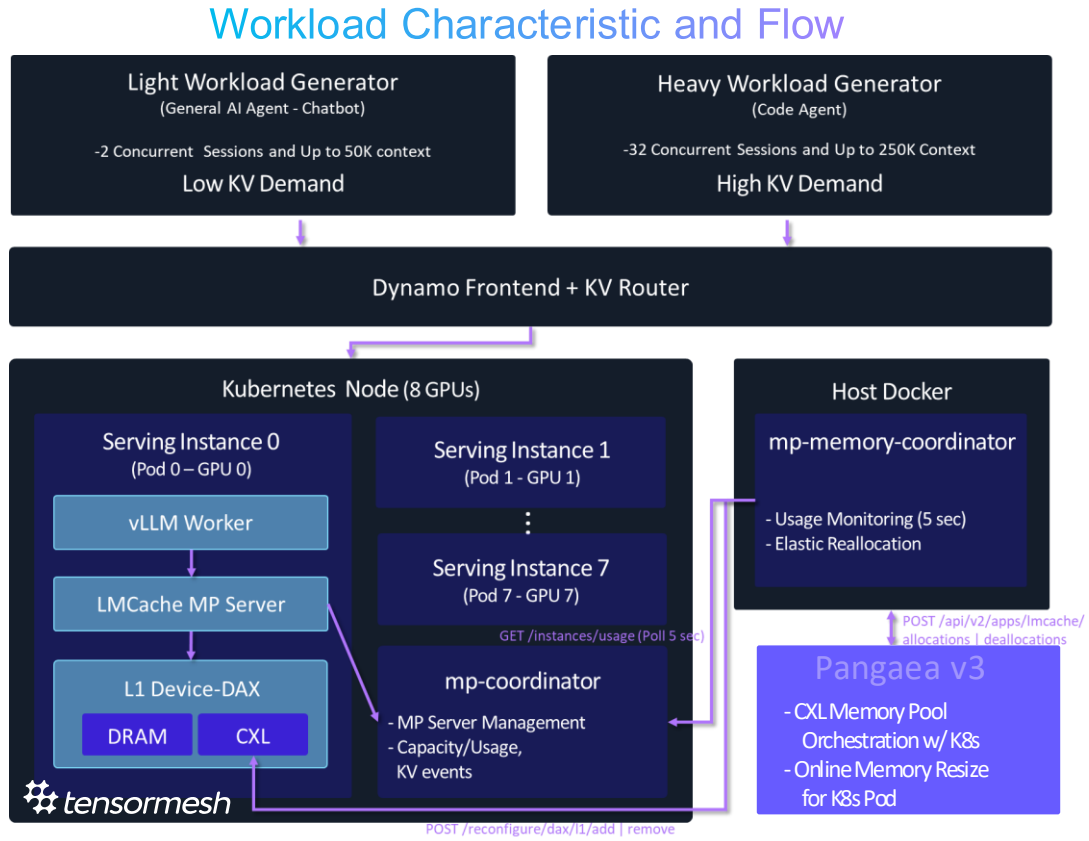




CXL DCD enables elastic memory allocation

- Samsung Pangaea orchestrated CXL memory pool for delivering KV Cache to multiple LLM instances via LMCache

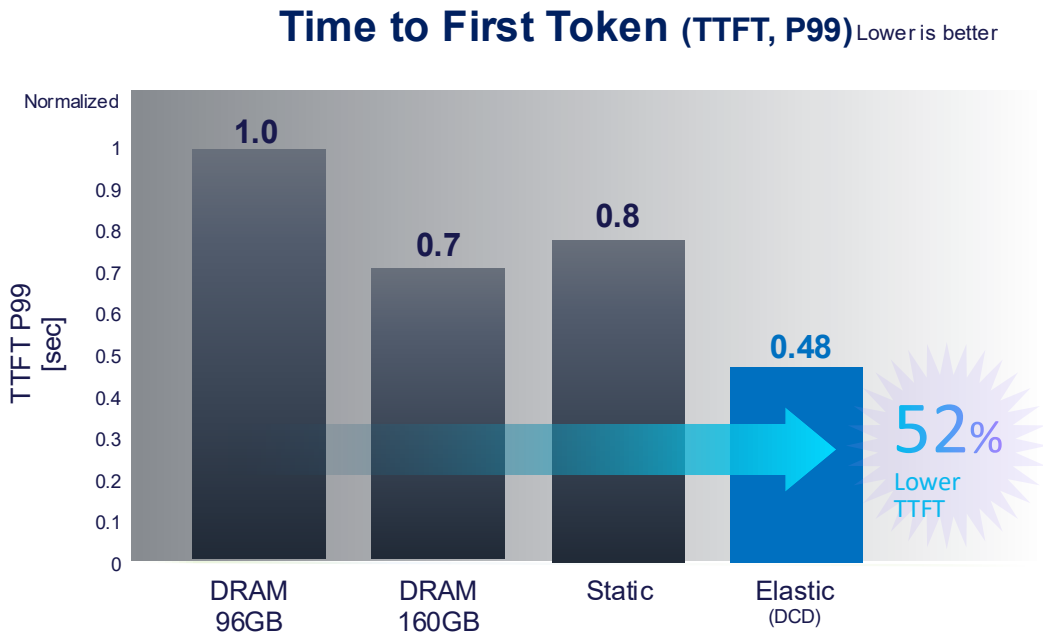


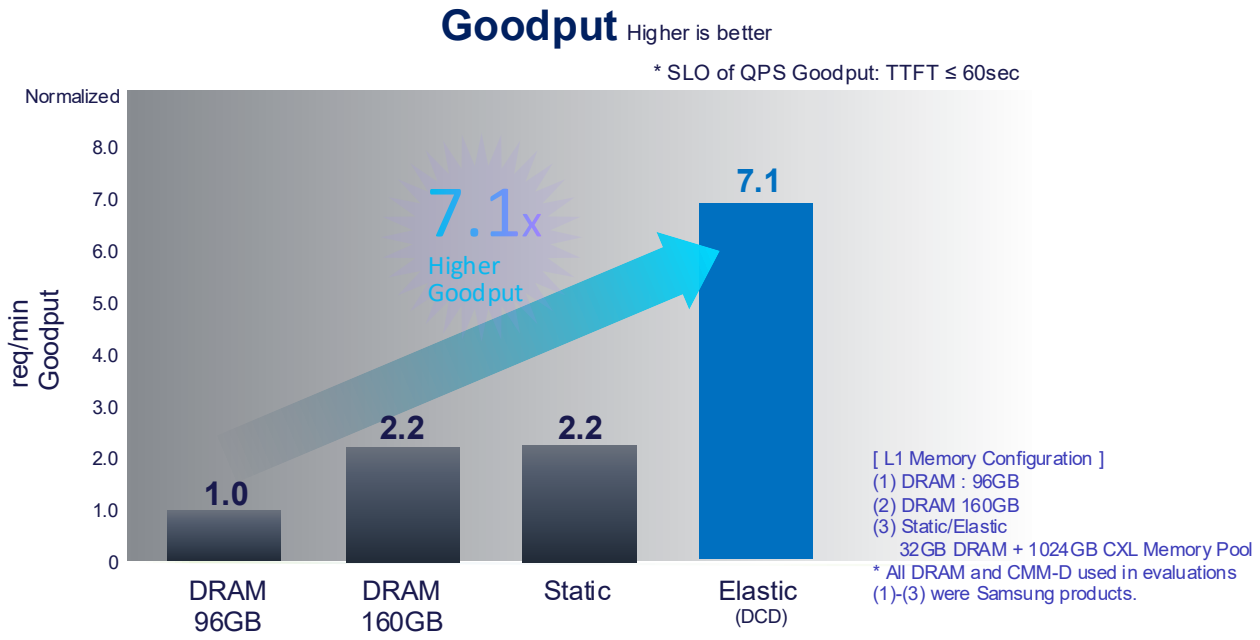


DCD Emulation: Reproducing CXL DCD Semantics on Today's Hardware
Dynamically add and release host-visible capacity per tenant, ahead of native CXL 3.x DCD platform availability

Performance of Elastic Memory Allocation

- 52% lower TTFT and 7.1x higher goodput across three LMCache storage-backend configurations





Static vs. Elastic

Elastic allocation reduces P99 TTFT and sustains higher QPS with dynamically resizable pooled memory

Elastic memory provisioning improves tail-latency resilience and End-to-End LLM serving efficiency

Key takeaway & Call to action

Performance

Long-context TTFT drops sharply when KV cache is reused from the CXL pool instead of recomputed

Efficiency

Stranded memory returns to the pool — cluster-level utilization rises, DRAM over-provisioning falls

Path to DCD

Today's DCD emulation is a working preview of native CXL 3.x dynamic capacity on next-gen platforms

Ecosystem collaboration with the OCP CMS:

Identify the fabric management requirements necessary for the CXL CMS and develop an interoperable fabric management schema and reference implementation.

Ecosystem collaboration with the LMCache framework:

a CXL-pool backend for KV-cache offload and cross-instance reuse

Connecting an open-source LLM serving stack to disaggregated CXL memory.

SERVER:
COMPOSABLE MEMORY
SYSTEMS (CMS)

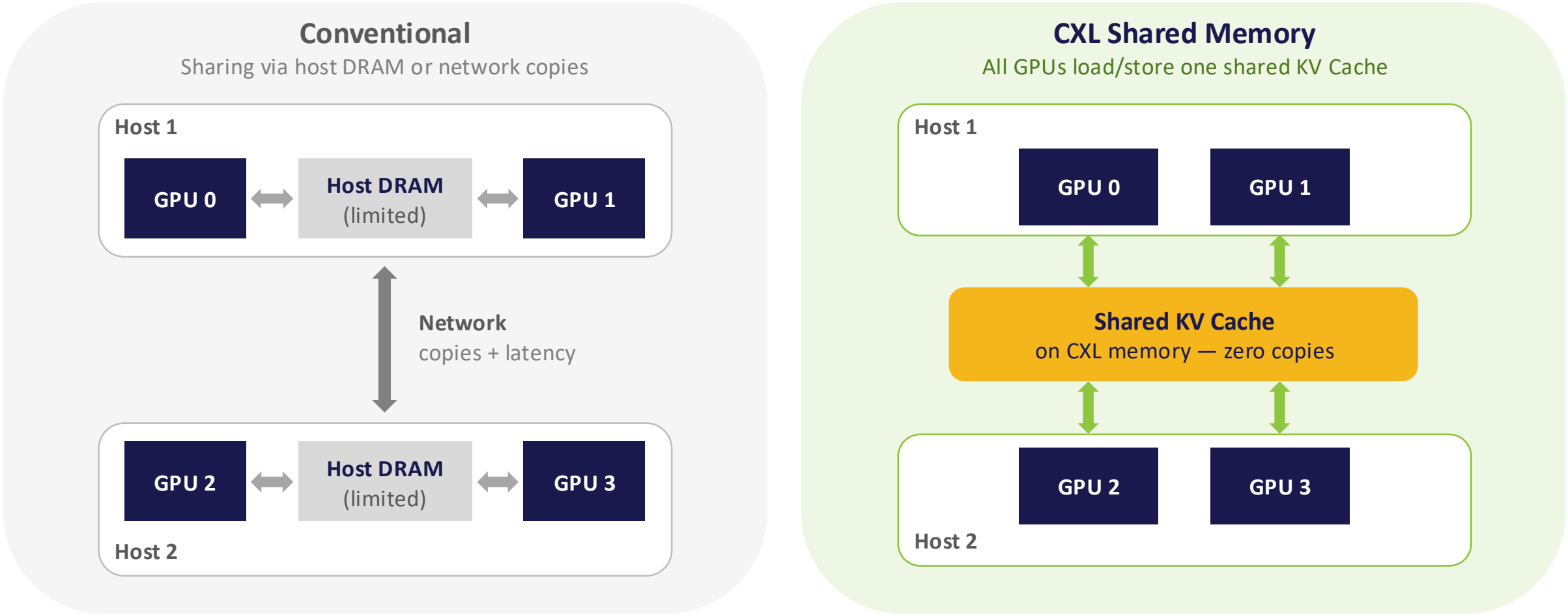


Scaling KV Cache with CXL Shared Memory and LMCache

Harry Kim

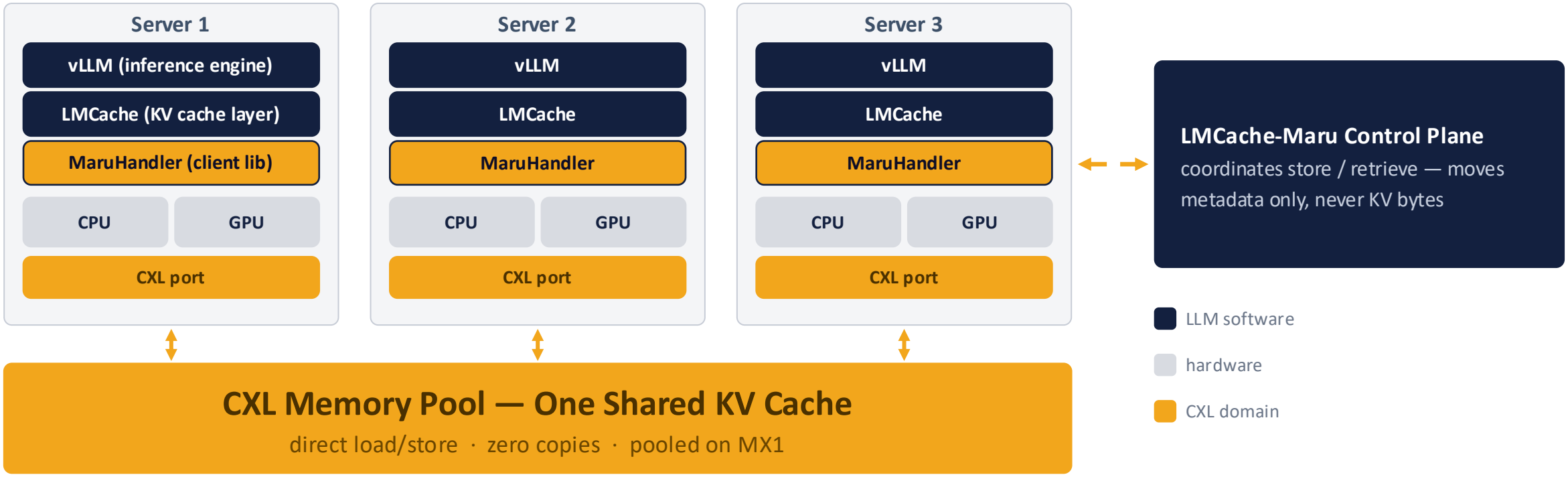
XCENA

Zero-Copy Memory Sharing Across GPUs



Access a shared CXL memory layer rather than copying KV Cache across GPU and host boundaries

Maru: Don’t Move KV Cache, Share the Data



Zero-Copy Sharing
No staging buffers, no serialization

Scales with Context
No KV payload fan-out over the network

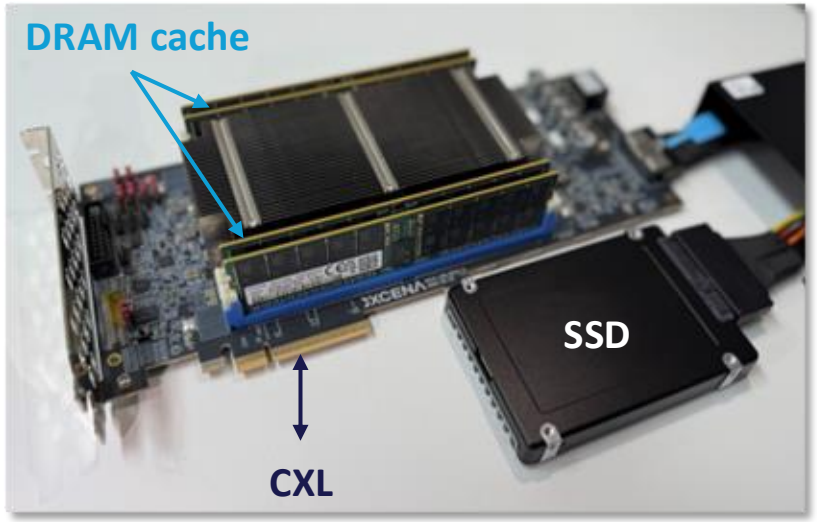
Higher Utilization
Replaces per-instance copies

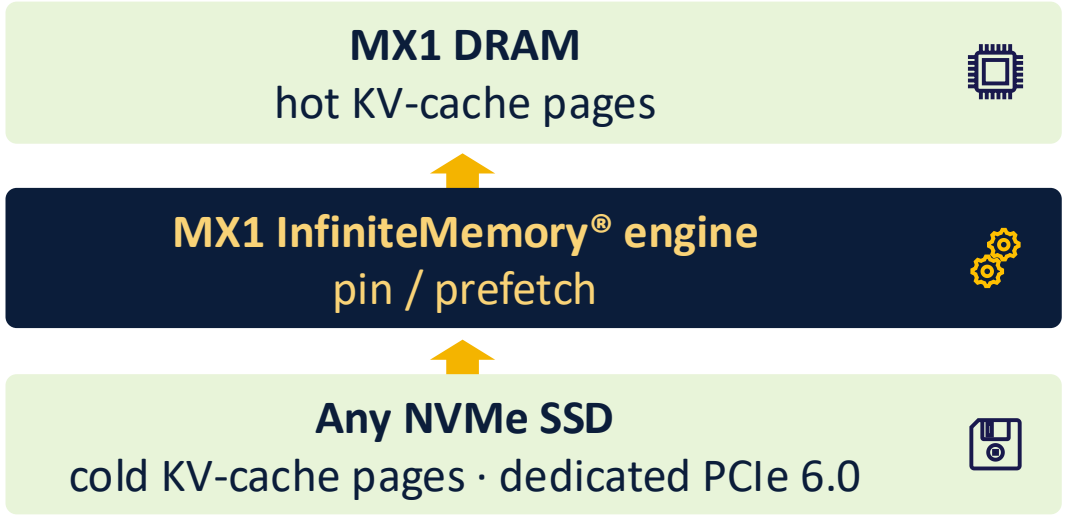
Lower Energy
Bulk transfers eliminated

Open-source: https://github.com/xcena-dev/maru

InfiniteMemory®: Speed of DRAM, Capacity of SSD

- MX1's InfiniteMemory® provides KV Cache reservoir with Capacity of SSD Storage, Performance of DRAM.
- Cold pages live on SSD; MX1 prefetches predictable KV-cache pages into DRAM before the GPU needs them

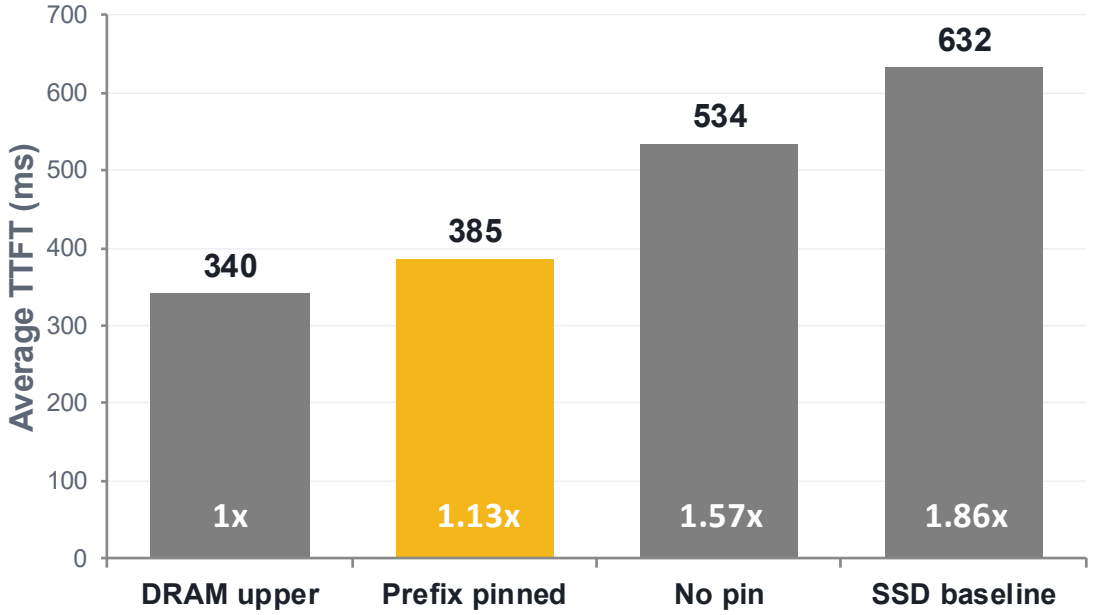




InfiniteMemory® applied to KV Cache

Pinned prefix ≈ DRAM

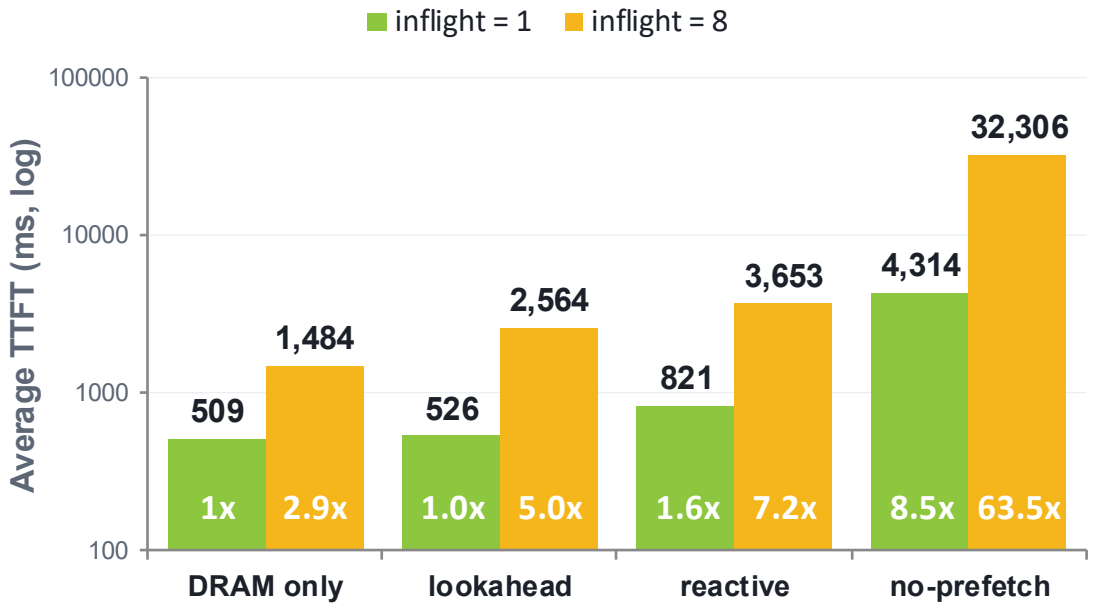
Queries against the shared document



- Pinned prefix stays resident, so query TTFT tracks DRAM (1.13×).
- Unpinned drops to 1.57×; raw SSD to 1.86×. Repeated prefix reads are eliminated.

Lookahead prefetch

Warm sessions, inflight 1 and 8



- Lookahead prefetch exploits device under-utilization to deliver DRAM-resident performance.
- With the right DRAM capacity and prefetch policy, storage overhead is effectively eliminated.

** Llama-3.1-8B, LV-Eval long-doc QA, vLLM-0.23.0 + LMCache-0.4.8 + Maru, NVIDIA RTX 6000 Pro

Bridging Granularity Mismatch

Memory semantics over storage media needs “glue” to bridge the gap.

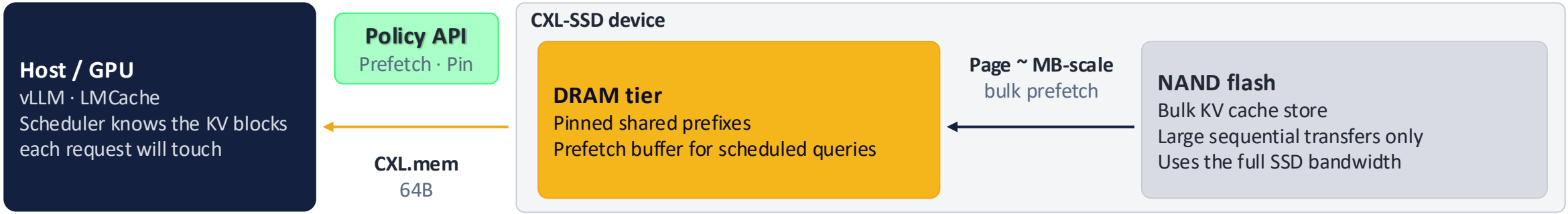
Problem — the granularity mismatch

- CXL works on **64B cache lines**; NVMe / NAND works on **4–16KB pages**
- 64B × 1K outstanding CXL commands = only 64KB in flight SSD
- 1K CXL commands cannot saturate a 10+GB/s SSD — and deep queues hurt QoS
- 64B random access is painful for an SSD

Why KV cache makes it tractable

- KV cache blocks are large: hundreds of KB to tens of MB
- Access pattern is deterministic — known when a query is scheduled
- **Prefetch** moves KV blocks SSD → DRAM at schedule time, ahead of access
- **Pin** keeps shared prefixes resident in DRAM, never round-trip to SSD

Read path

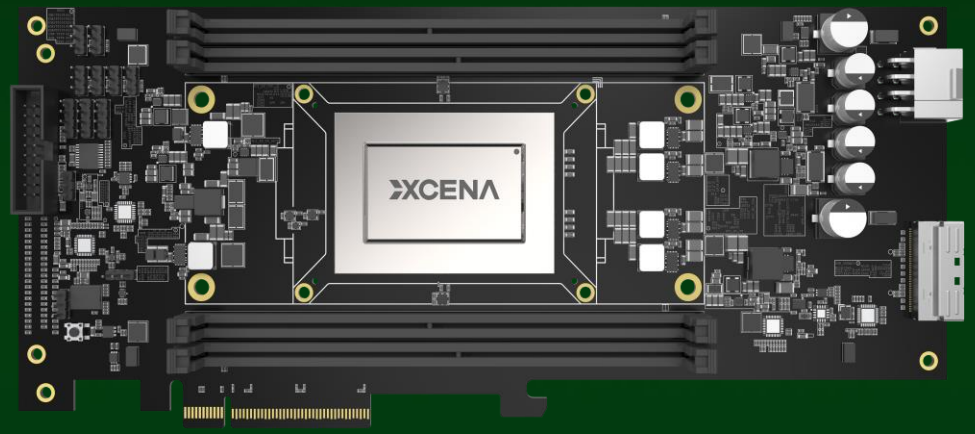


Schedule-time prefetch hints help move MB-scale KV blocks NAND → DRAM before the GPU accesses them.

Takeaways and Call to Action

- KV cache sharing on CXL memory is demonstrated and available today. Memory semantics with no RDMA copies deliver efficient storage and low latency.
- On top of this, a CXL+SSD hybrid offers even more value: cost-efficient storage at near-DRAM performance.
- The OCP CMS community would be a great venue to discuss and standardize the policy API for emerging memory devices with these new characteristics.
- References
  - Maru + LMCache Integration: https://docs.lmcache.ai/kv_cache/storage_backends/maru.html
  - InfiniteMemory®: https://xcena.com/infinitememory
  - XCENA SDK: https://www.xcena.com/sdk_overview

Thank You!



MX1 CXL Computational Memory

XCENA
Xcelerate Your Intelligence

Booth #A2

SERVER:
COMPOSABLE MEMORY
SYSTEMS (CMS)



Computational Memory System for Agentic AI

Jinin So, Arifur Rahman, Harry Kim

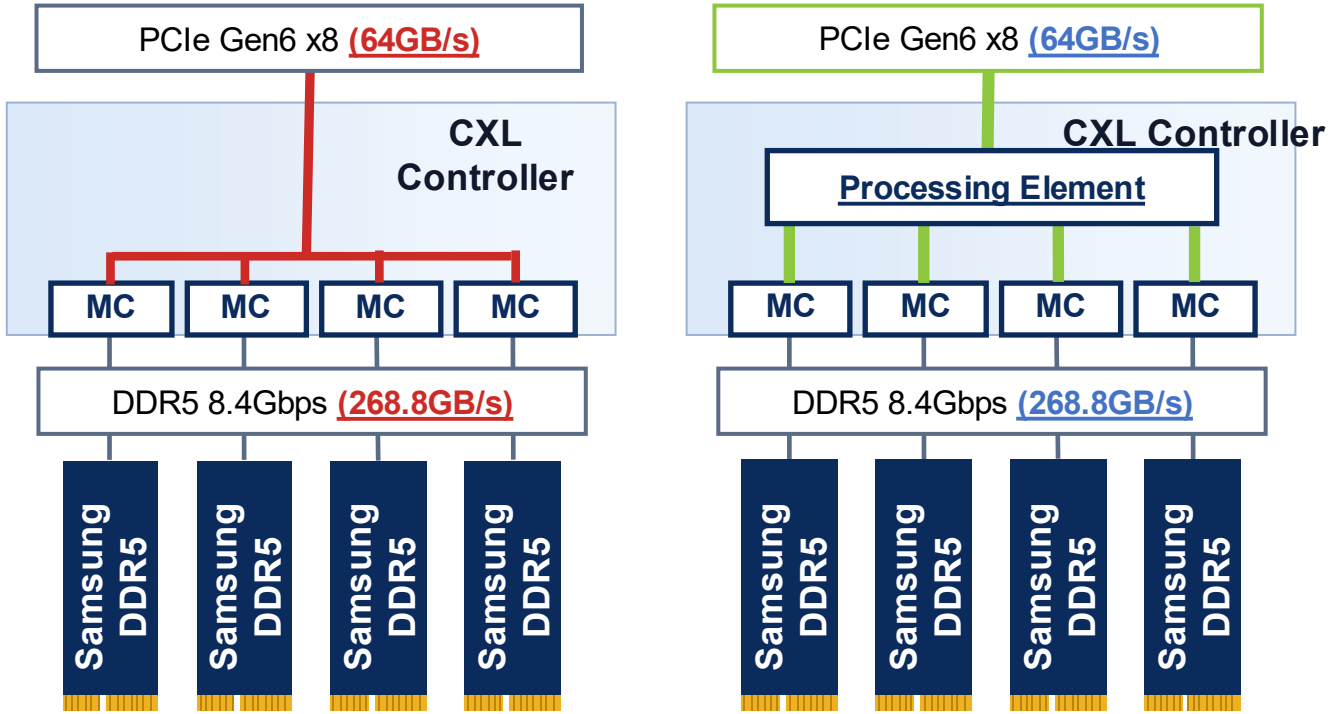
Samsung, Marvell, XCENA

Samsung CXL-PNM Research (with Marvell & XCENA)

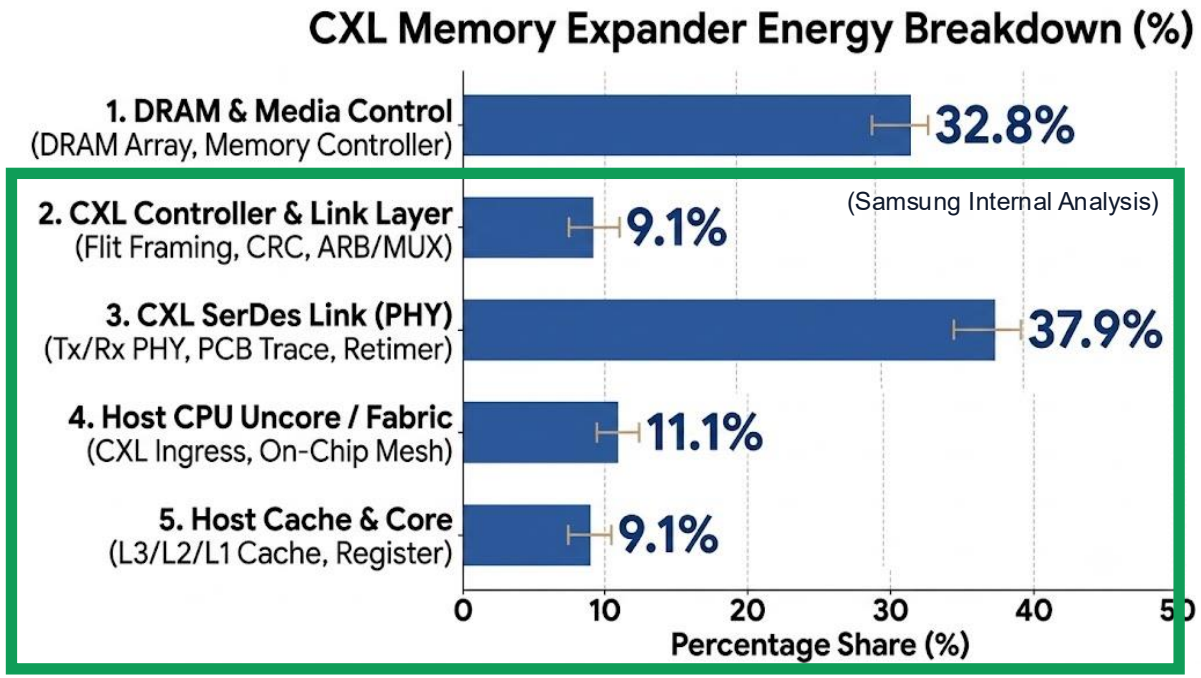
(Processing-Near-Memory)

- Maximize Utilization of **huge DRAM Bandwidth** behind CXL Interconnect.
- Compute In-place and **ONLY Reduced Data Transferred**. ➔ Data transfer **energy reduction**.

Maximize DRAM BW behind CXL



Minimize Data Transfer Energy



Data transfer energy(pJ/bit) between CXL Controller to Host (67.2%) can be minimized with PNM.

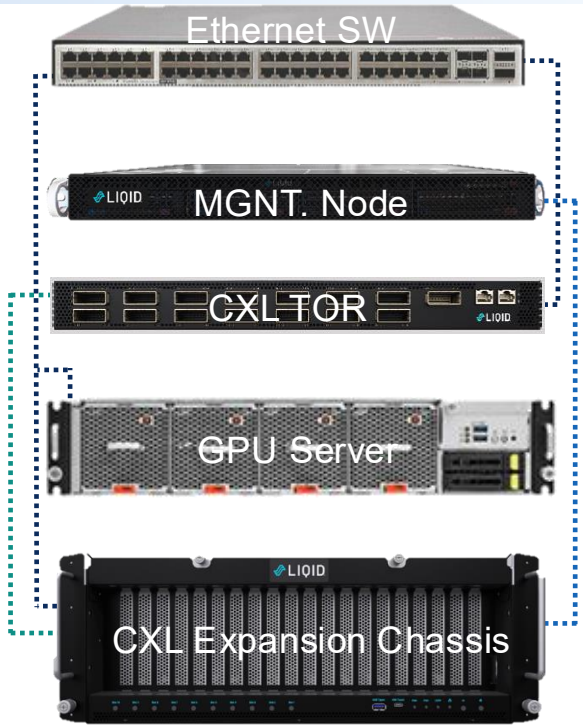
Samsung CXL Computational Memory System

- GPU Servers can share this memory system through CXL memory semantic directly.
- LLM KV cache offloading(pooling, sharing) with memory semantic, RAG Vector DB
- Memory Capacity : **20TB**, Memory Bandwidth : **2.7TB/s**

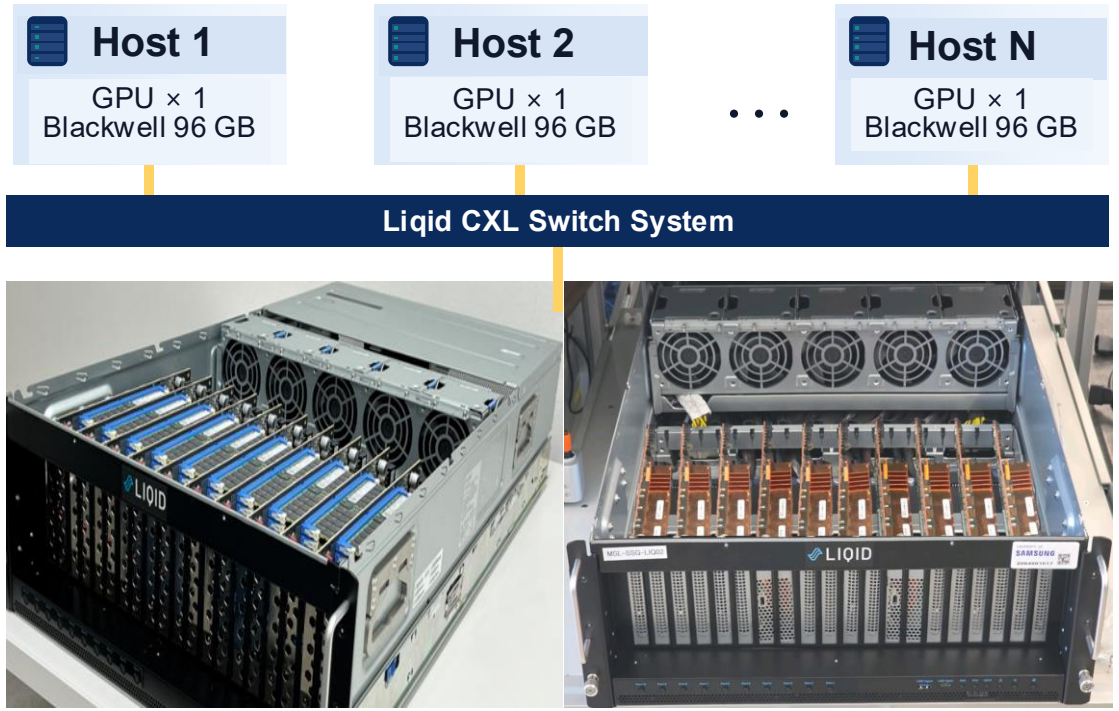
Rack-Scale CXL Appliance



Building Blocks



System Topology

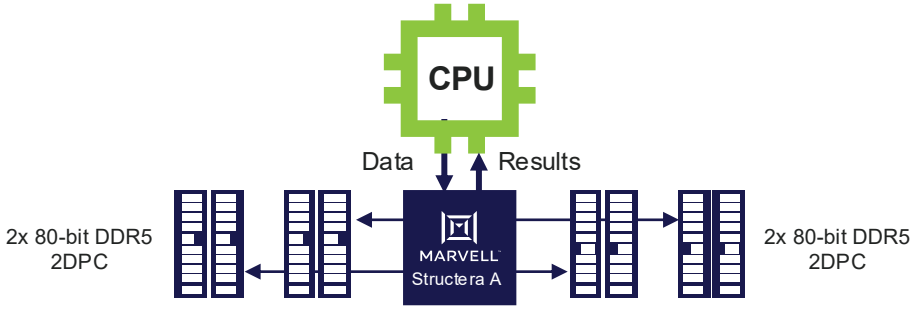


Marvell's Near-Memory Compute with V2 Core Integration

Structera A

Near-memory compute accelerators





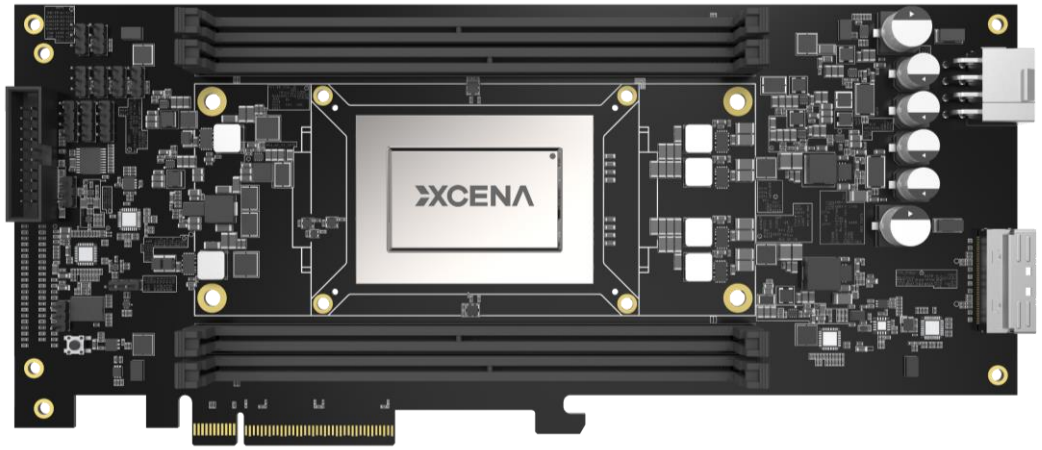
PCIe **5.0 1x16 | 2x8** CXL **2.0 | DDR5 4-ch**	**16 ARM V2 Demeter cores at 3.2GHz**	**200GB/s DDR5** Internal memory BW	**LZ4 compression** **2-3x** memory capacity
5nm	**Server-class** cores	**95W** at 3.0GHz	Up to **4TB** | AES 256

AI inference | **DLRM** | **HPC**

Xcena's Near-Memory Compute with RISC-V

MX1: Expand, Extend, Compute

Memory is intelligence. AI workloads are bound by memory capacity, bandwidth, and power efficiency rather than raw compute.



CXL Memory Expansion

- 4ch DDR5 8400MT/s, up to 2TB
- CXL 3.2 Type 3, PCIe Gen6
- Chip-kill RAS

Infinite Memory Extension

- SSD on MX1 root port = CXL memory
- Host sees full SSD capacity
- DRAM as cache tier

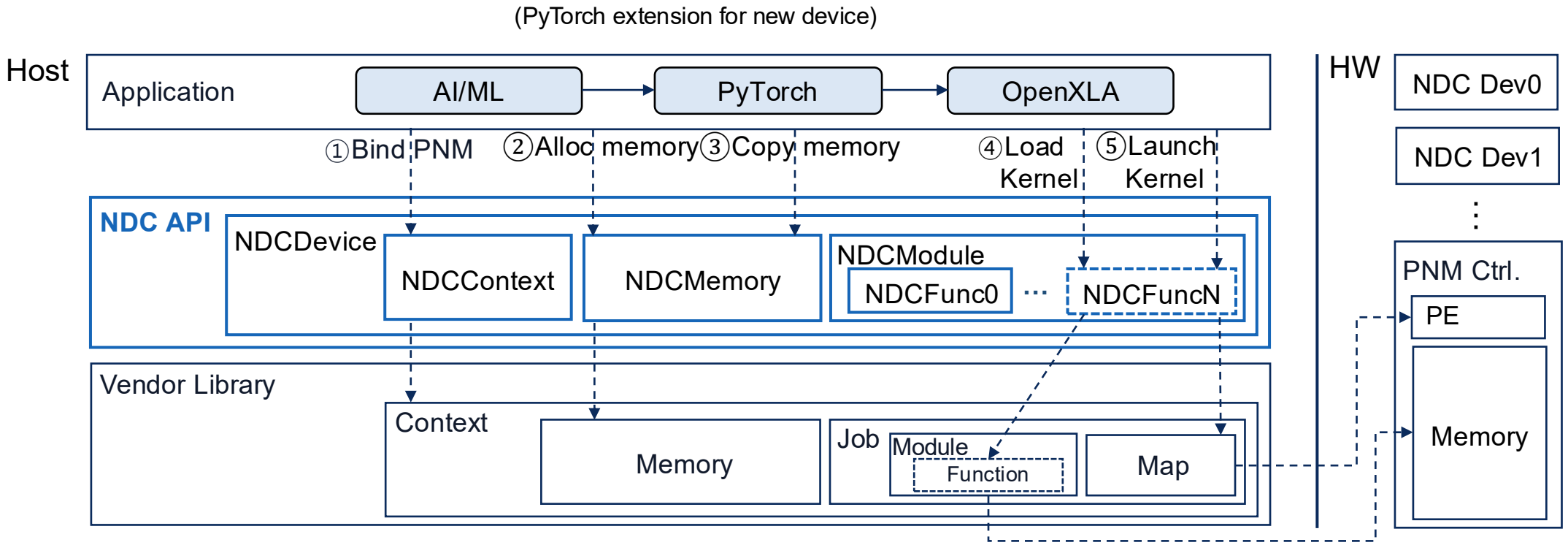
Near-Memory Compute

- 3,072 custom RISC-V cores
- User-defined kernels
- SDK available — xcena.com/SDK

Samsung SW Runtime for PNM: NDC API

(Near-Data-Computing)

- NDC API[1] : Samsung vendor-agnostic library for programming PNM devices
- **Open-sourced** via OCP FTI DCC, released to accelerate Near-Data-Computing ecosystem.
- PyTorch integration in progress via OpenXLA and PrivateUse1 with **no code change**.

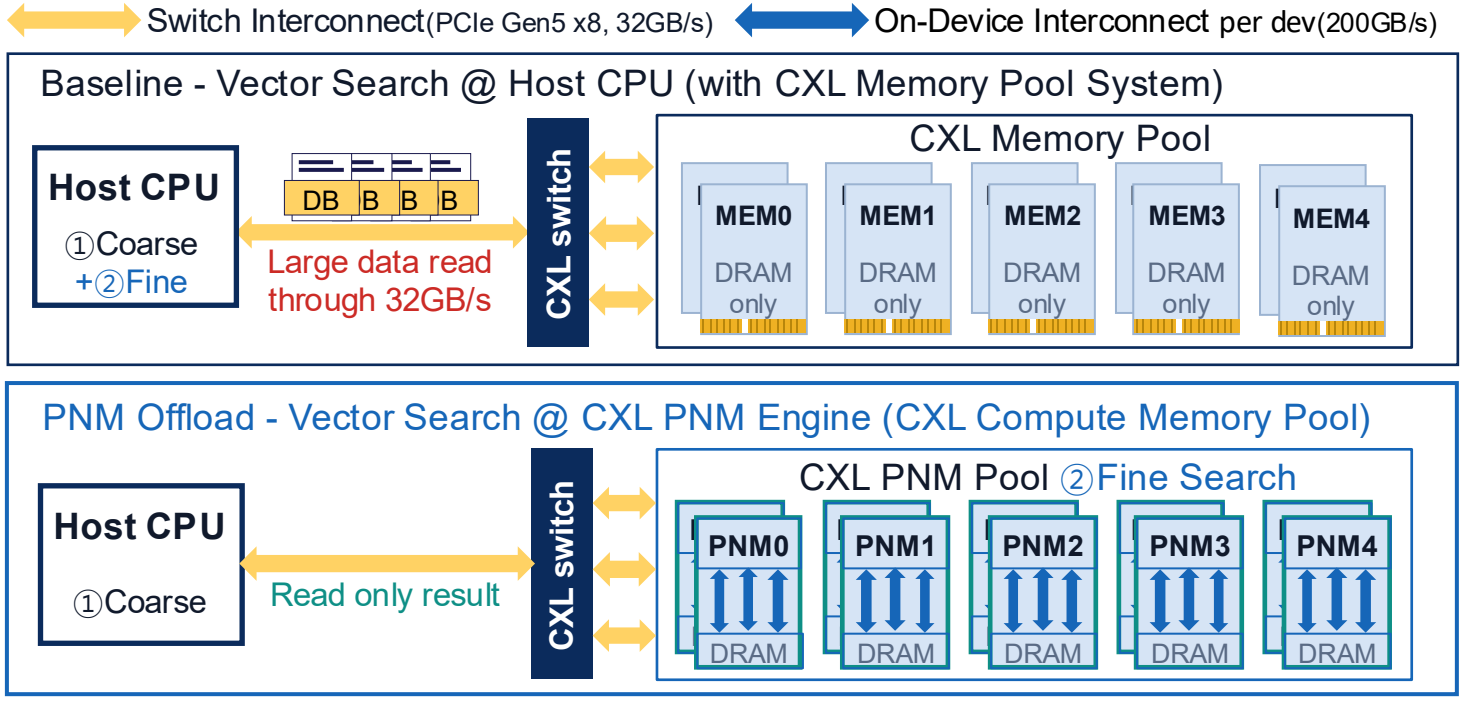


1) OCP NDC API github : https://github.com/opencomputeproject/samsung-ndc

AI Use Case 1 - RAG Vector Search on CXL-PNM

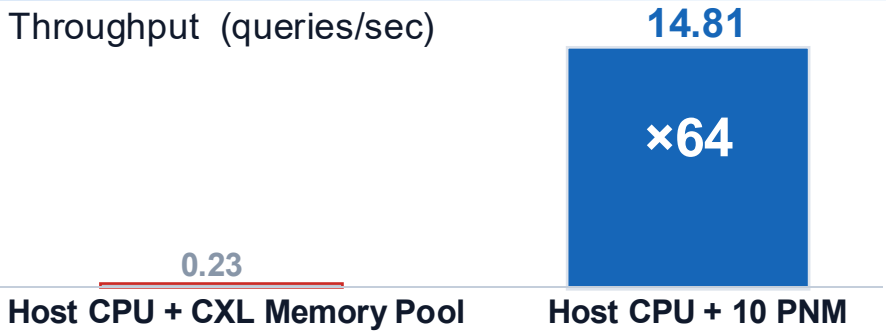
- IVF Flat indexing using FAISS Library, offloading L2-kNN computations to CXL PNM devices
- L2-kNN distance computation is memory-bandwidth-bound

FAISS Index Search Workflow



Index: IVF-Flat (FAISS), 512M total vectors, nlist=6144, nprobe=1920
Dataset: Laion5B, convert to 1024-dimension vector using CLIP-ViT-H-14, sample to 4.5TB vectors

10 PNM devices deliver 64× higher QPS



Effective Read BW utilization
- Host CPU Read BW: 26 GB/s
- 10x PNM aggregate BW: 1625GB/s

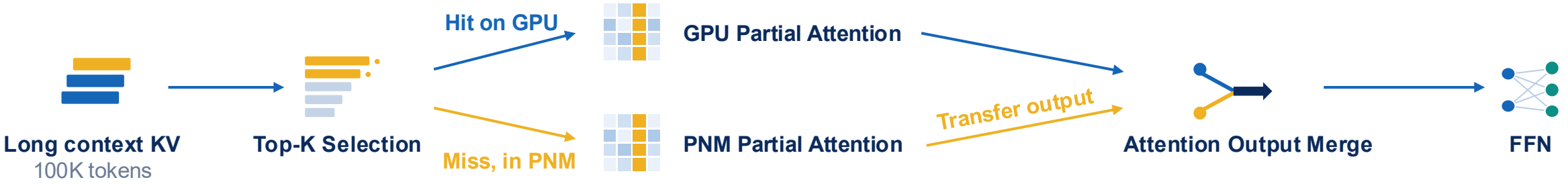
65× better query-per-energy



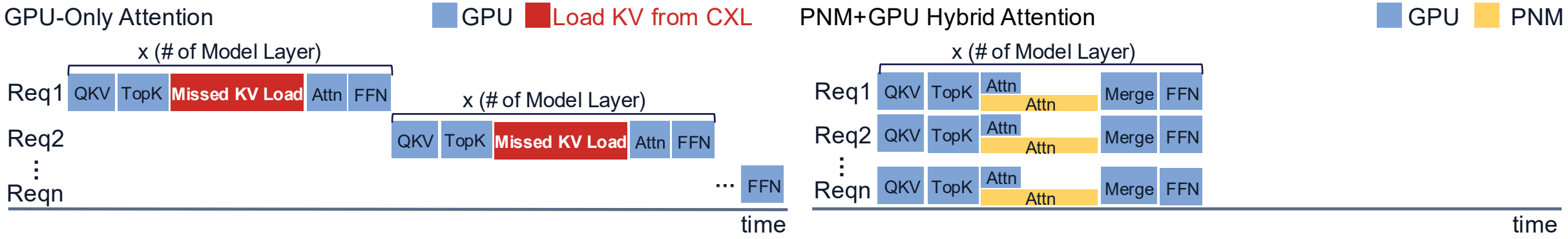
AI Use Case 2 - GPU + PNM Hybrid Attention

- Normally, KV cache Transfer time > calculation time on GPU
- In decode stage, instead of loading all KV pages, delivering only attention result calculated by PNM

Hybrid Attention - Hit Pages on GPU, Miss Pages on PNM in decode attention



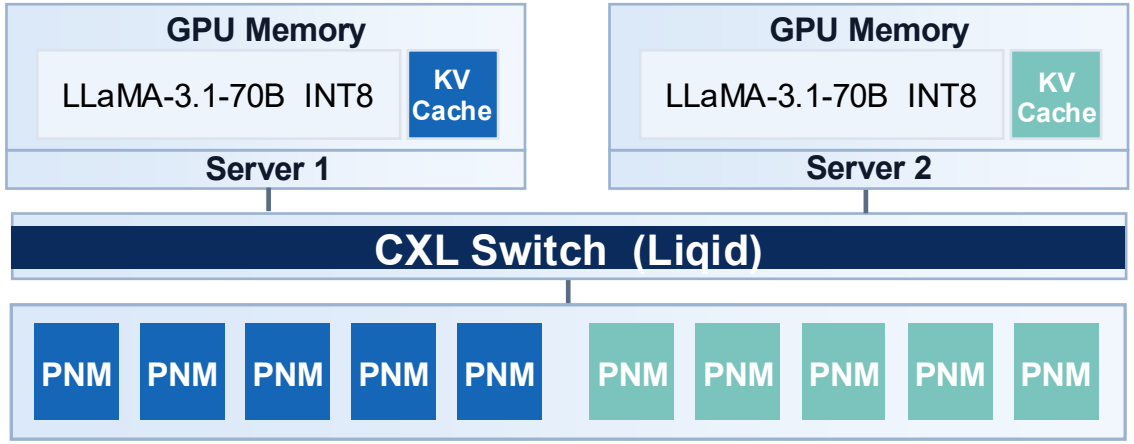
Decode Timeline - GPU-Only Attention vs Hybrid Attention (batch size=n)



AI Use Case 2 - GPU + PNM Hybrid Attention

- KV cache Data transfer time is **much longer** than PNM compute time @ multi-batch workload.
- At 100K context, using the PNM pool improves throughput by **3.35x** and energy efficiency by **3.84x**.

Experimental Setup (2 servers, 5 PNM / server)



Model : LLaMA 3.1 70B, DP=2, with INT8 quantized weights(70GB)

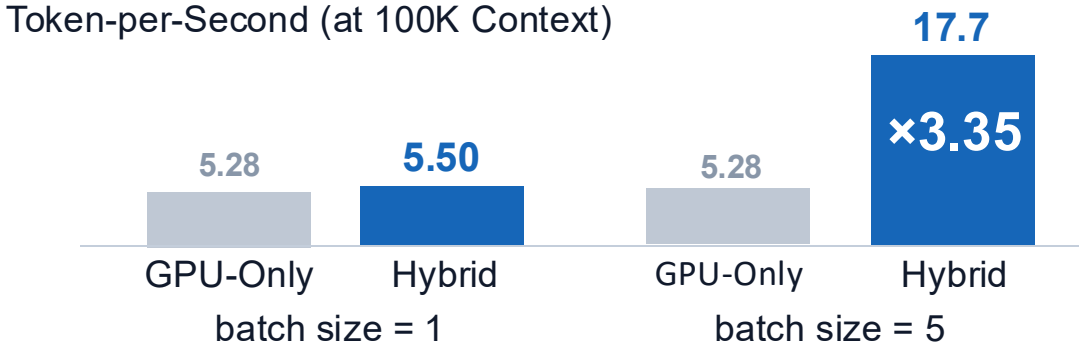
Metrics : 100K context decode step latency

Context : 100K tokens per request

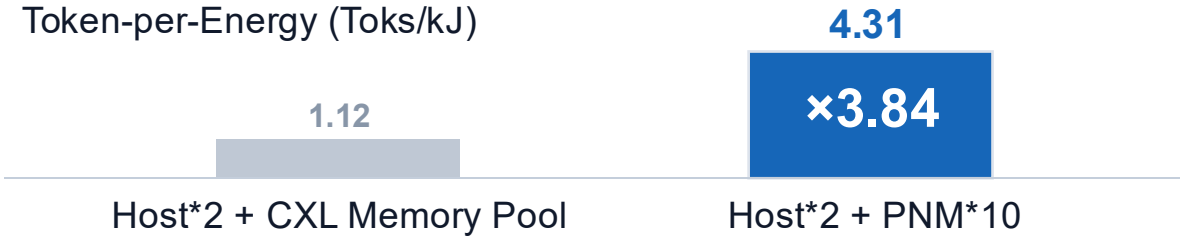
GPU : NVIDIA RTX PRO 6000 Blackwell (96 GB)

GPU KV Cache Size : 8GB per GPU

Hybrid attention gives 3.35× throughput at 100K context



3.84× better token-per-energy



Takeaways and Call to Action

- CXL based CMS(pooling/sharing) can be one promising solution to solve KV cache explosion in the view of memory capacity.
- However, the bandwidth & energy challenges of interconnect should still be solved for scalable inference infrastructure.
- PNM(Process-Near-Memory) can relieve the stress of KV cache data transfer with optic technology.
- Let’s build PNM ecosystem including SW stack @ CMS & DCC-FTI.